



CSC1029 OO Programming

Lecture 4 – Programs with Methods



Case Study: From Design to Implementation (1)

Design and implement a program called “WordWrap” which:

1. asks the user to input a word from the keyboard
2. uses the console to display the word according to the following pattern:

Enter a single word: Banana

```
Banana
a      n
n      a
a      n
n      a
ananaB
```

Using pen/paper:

- Identify what data is required
- Produce an outline design

Case Study: From Design to Implementation⁽²⁾

Need to display a message

Enter a single word:

Banana

Need to read a String

Banana

Display the word as it is

a n

n a

a n

n a

display letters going forward
display letters going backwards

ananaB

Display the word in reverse

Recap – the story so far



primitive data

statement sequences

selection (making decisions)

iteration (loops)

methods

objects



Recap – Case Study: 'WordWrap' ⁽¹⁾

display message

read a word

output the word

output middle lines

output word in
reverse

```
// display a message & read word
```

```
System.out.print("Enter a single word: ");
```

```
String word = input.nextLine();
```

```
# display a message & read word
```

```
word = input("Enter a single word:  
")
```

```
// display a message & read word
```

```
Console.Write("Enter a single word: ");
```

```
String word = Console.ReadLine();
```



Recap – Case Study: 'WordWrap' ₍₂₎

display message

read a word

output the word

output middle lines

output word in

reverse

```
// output the word
```

```
System.out.println(word);
```

```
# output the word
```

```
print(word)
```

```
Console.WriteLine// output the word
```

```
Line(word);
```



Recap – Case Study: 'WordWrap' ⁽³⁾

output middle lines

```
// get the length of word
int len = word.length();

// len-2 lines to display
int numLines = len - 2;

// initial index for 1st character
int index1 = 1;
// initial index for 2nd character
int index2 = len-2;

// gap between characters
int gap = len-2; // gap between chars
```

index1



"b a n a n a"

0 1 2 3 4 5

index2



Recap – Case Study: 'WordWrap' ⁽⁴⁾

output middle lines

```
// get the length of word
int len = word.length();

// len-2 lines to display
int numLines = len - 2;

// initial index for 1st character
int index1 = 1;
// initial index for 2nd character
int index2 = len-2;

// gap between characters
int gap = len-2; // gap between chars
```

```
for(int count = 1; count<=numLines; count++)
{
    // output 1st character
    System.out.print(word.charAt(index1));
    index1++; // move index1 up 1

    // output spaces
    for(int space=1; space<=gap; space++) {
        System.out.print(" ");
    }

    // output 2nd character
    System.out.println(word.charAt(index2));
    index2--; // move index2 down 1
}
```


Recap – Case Study: 'WordWrap' ⁽⁵⁾

Output in reverse

```
// get the length of word
int len = word.length();

// len-2 lines to display
int numLines = len - 2;

// initial index for 1st character
int index1 = 1;
// initial index for 2nd character
int index2 = len-2;

// gap between characters
int gap = len-2; // gap between chars
```

```
for(int count = len-1; count>=0; count--) {
    System.out.print(word.charAt(count));
}
System.out.println();
```

Recap – Case Study: 'WordWrap' ⁽⁶⁾

output middle lines

```
# get the length of word
len = len(word)

# len-2 lines to display
numLines = len-2

# initial index for 1st character
index1 = 1
# initial index for 2nd character
index2 = len-2

# gap between characters
gap = len-2
```

```
for count in range(numLines):
    # output 1st character
    print( word[index1], end="" )
    index1 = index1 + 1      # move index1 up 1

    # output spaces
    for space in range(gap):
        print(' ', end="");

    # output 2nd character
    print( word[index2] )
    index2 = index2 - 1      # move index2 down 1
```

Recap – Case Study: 'WordWrap' ⁽⁷⁾

output middle lines

```
// get the length of word
int len = word.length();

// len-2 lines to display
int numLines = len - 2;

// initial index for 1st character
int index1 = 1;
// initial index for 2nd character
int index2 = len-2;

// gap between characters
int gap = len-2; // gap between chars
```

```
for (int count = 1; count <= numLines;
count++)
{    // for each line
    // output 1st character
    Console.Write(word[index1]);
    index1++;    // move index1 up 1

    // output spaces
    for (int space = 1; space <= gap;
space++)
    {
        Console.Write(" ");
    }

    // output 2nd character
    Console.WriteLine(word[index2]);
    index2--;    // move index2 down 1
}
```



What's to come today?

- what are methods?
- method structure
- methods that return data (functions)
- methods with parameters
- example: simple arithmetic
- example: 'WordWrap' revisited

What are methods?

Methods are **named** statement sequences that, once defined, can be repeatedly executed as required.

Methods support **code reuse** and program **clarity/structure**.

Methods are also used in the definition of **object behaviour** (as we will see later)

```
public class MyProgram {  
    public static void main(String args[]){  
        calc1(); //method call  
    }  
  
    public static void calc1() {  
        calc2(); //method call  
    }  
  
    public static void calc2() {  
        System.out.print(2+2);  
    }  
}
```

Methods that return data

Methods defined using the `void` keyword – perform a task, but will not return any value.

Methods which return data must specify the `return type`.

Such methods must have an associated `return statement`.

```
public class MyProgram {  
    public static void main(String args[]){  
        calc1(); //method call  
    }  
  
    public static void calc1() {  
        int value = calc2();  
    }  
  
    public static int calc2() {  
        return 3 + 2;  
    }  
}
```

Methods that use parameters

Data may be shared with methods via parameters.

Methods may be defined with a formal parameter list.

These method must be called with an actual parameter list which matches the formal list.

```
public class MyProgram {  
  
    public static void main(String args[]) {  
  
        int num = sum(2, 3);  
  
    }  
  
    public static int sum(int a, int b) {  
  
        return a + b;  
    }  
  
    public static int mult(int a, int b) {  
  
        return a * b;  
    }  
  
}
```

Code Example



Calculator.java

CalcTest.java

Case Study: 'WordWrap' Revisited

The first version of 'WordWrap' is defined using the main method only.

The program can be organised to use multiple methods.

What's the point of doing this?

```
public class WordWrap {  
    public static void main(String args[]){  
        // how ?  
        // what's next ?  
        // have a go!  
    }  
  
}
```



What's Next?

Objects

using library objects

user-defined objects